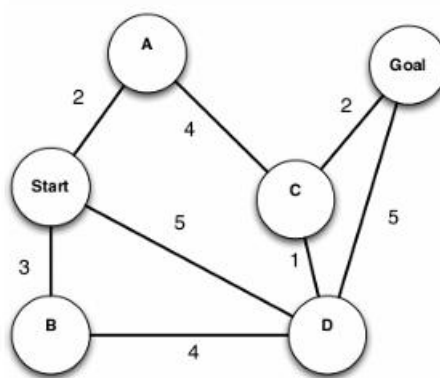


Theoretical Questions

1. For each of the graph search strategies, work out the order in which states are expanded, as well as the path returned by graph search. Break ties such that states with earlier alphabetical order are expanded first.



- a) Depth-first search
- b) Breadth-first search
- c) Uniform cost search.

2. Consider an undirected, connected graph with V vertices and E edges. Assume that BFS (Breadth-First Search), DFS (Depth-First Search), and UCS (Uniform Cost Search) are used to explore this graph starting from an initial node.

What are the worst-case time and space complexities of BFS and DFS in terms of V and E ?

How does the complexity change if the graph is dense ($E=O(V^2)$) versus sparse ($E=O(V)$)?

Complete the table and provide an explanation for your reasoning behind each entry.

	Sparse				Dense			
	Time		Time		Time		Space	
	Best	Worst	Best	Worst	Best	Worst	Best	Worst
BFS								
DFS								
UCS								

Comparison and Practical Impact:

- Which of BFS, DFS, and UCS would be the most memory-efficient for searching in an extremely large but sparse graph, and why?
- In which cases would UCS perform worse than BFS in terms of time complexity?

3. For each of the following assertions, say whether it is true or false and support your answer with examples or counterexamples where appropriate.

- An agent that senses only partial information about the state cannot be perfectly rational.
- There exist task environments in which no pure reflex agent can behave rationally.
- There exists a task environment in which every agent is rational.
- The input to an agent program is the same as the input to the agent function.
- Every agent function is implementable by some program/machine combination.
- Suppose an agent selects its action uniformly at random from the set of possible actions.
- There exists a deterministic task environment in which no agent can be perfectly rational.
- It is possible for a given agent to be perfectly rational in two distinct task environments.
- Every agent is rational in an unobservable environment.
- A perfectly rational poker-playing agent never loses.

Programming Questions

1. Advanced Depth-First Search (DFS) with State Pruning

Problem Description:

In this task, you are given a state space where each state has a set of successor states. However, some successor states might lead to dead-ends, and repeatedly revisiting states (looping) can cause infinite recursion and inefficient search.

Your task is to implement Depth-First Search (DFS) with state pruning. Specifically, you should:

1. Avoid revisiting previously expanded states (loop prevention).
2. Use a memory-efficient technique to track visited states without storing all of them in memory (i.e., using disk-based storage or a limited-size data structure).

Tasks:

- Implement a DFS algorithm that uses pruning to avoid revisiting states.
- Use disk-based storage or an efficient data structure to track visited states.
- Compare the time complexity and space complexity of this implementation versus a naive DFS that stores all visited states in memory.
- Discuss the challenges of state pruning and the trade-offs between time complexity and memory usage.

2. Iterative Deepening Depth-First Search (IDDFS) with Variable Depth Limits

Problem Description:

You need to implement Iterative Deepening Depth-First Search (IDDFS) for a state space that is very large, where the maximum depth of the solution is unknown. In addition, the depth limits for each iteration should adapt dynamically based on the current exploration progress (i.e., increase depth limits intelligently rather than arbitrarily).

This adaptation is crucial in a scenario where you're trying to solve a puzzle (e.g., a sliding puzzle), where the solution might be close to the starting state in some cases and farther in others.

Tasks:

- Implement IDDFS where the depth limit for each iteration is increased dynamically based on feedback from previous iterations (e.g., how much of the state space was explored).
- Add dynamic path reconstruction after reaching the goal state (you must trace back the path efficiently without storing the entire search tree).
- Test this algorithm on a sliding puzzle (e.g., 8-puzzle) where the depth of the solution varies.
- Discuss the advantages and disadvantages of using dynamic depth limits in IDDFS compared to traditional fixed-depth DFS and BFS.

3. Uniform Cost Search (UCS) with Dynamic Cost Updates

Problem Description:

In a dynamic environment where edge costs are constantly changing (e.g., due to traffic, weather, or other conditions), you must implement Uniform Cost Search (UCS) to find the shortest path from a starting node to a goal node in a graph. However, the edge costs are not static, and they may change as the search progresses.

Your UCS implementation should:

1. Account for dynamic edge cost updates during the search process.
2. Ensure that the optimal path is found even if the edge costs change mid-search.
3. Implement an efficient method to recalculate paths when costs change, so that previously explored nodes are not unnecessarily re-explored.

Tasks:

- Implement UCS that can handle dynamic edge cost changes.
- Design a mechanism that allows re-calculation of the optimal path without losing efficiency.

- Simulate a dynamic road network (e.g., a transportation system) where edge costs represent travel times, and the goal is to find the shortest path between two locations.
- Compare the performance of your UCS implementation when edge costs remain constant vs. when they change dynamically during the search.

4: Bidirectional Search in Large Graphs with Symmetric States

Problem Description:

You are tasked with implementing Bidirectional Search (BDS) for a large graph with symmetric states. The goal is to find the shortest path from the start state to the goal state by simultaneously searching from both the start and goal states. The challenge is that the graph has symmetry (i.e., the same states can be reached from both directions in similar patterns).

Your task is to:

1. Implement Bidirectional Search with state-space symmetry taken into account.
2. Ensure that your algorithm efficiently detects and exploits this symmetry to reduce the number of expanded nodes.
3. Handle cases where the graph is too large to fit entirely in memory, using a memory-efficient approach.

Tasks:

- Implement Bidirectional BFS and ensure it works efficiently with symmetric state spaces.
- Use a shared search frontier to ensure that both searches meet at the same node without redundant work.
- Demonstrate your solution with a large puzzle (e.g., 15-puzzle or graph-based problem) to show how the symmetry of the problem helps reduce the search space.
- Analyze the time complexity and space complexity of your bidirectional search compared to traditional unidirectional BFS.

Important Notes

- Academic Integrity Any form of academic dishonesty will result in a zero grade for this assignment.
- You are allowed to use tools like GPT to assist you in writing code or understanding the concepts, but it is crucial that you fully understand the code you write and how it works. You may be required to explain your implementation during an assessment.
- Code Quality: Your code should be well-documented and follow good programming practices.
- You need to implement the algorithms from scratch. Using the built-in functions and algorithms is not allowed.
- Provide a report in PDF format and explain your code and results.
- The name of the uploading file should be “Firstname_Lastname_StdNumber.zip” of all group members.
- Collaboration Policy: While discussions with peers are allowed, your submitted work must be original.
- Late Submissions: Late submissions may incur penalties as per course policy.
- If you have any questions about anything, feel free to ask in the course's group chat.

Good Luck!